AI Search Algorithms & Decision-Making

Scenario-Based Questions — Solutions

Scenario 1 — AI Drones for Flood-Relief Medicine Delivery

The drone must plan a route through a dynamic, partially-observable environment. It has a heuristic (straight-line/Euclidean distance to the target), partial real-time information about blocked roads, and variable movement costs caused by terrain and weather. This is a classic setting for comparing an uninformed-of-cost heuristic search (Greedy Best-First Search) with a cost-aware informed search (A*).

i. Behaviour of Greedy Best-First Search (GBFS)

GBFS expands the node that appears closest to the goal according to the heuristic h(n) alone, ignoring the actual path cost g(n) accumulated so far. In this scenario it would repeatedly choose the next drop-off waypoint whose straight-line distance to the destination is smallest, without considering how expensive or risky that leg of the journey actually is.

Strengths

- **Speed:** Because it only evaluates h(n), it expands very few nodes and reaches a solution quickly — valuable when medicines must be delivered with minimum delay.
- **Low memory/computation:** Useful for a resource-constrained onboard flight computer that must re-plan frequently.

Weaknesses under uncertainty

- **Sub-optimality:** GBFS can be lured toward a waypoint that looks close on a straight line but is actually cut off by floodwater or requires a long detour, producing long or costly real paths.
- **No safety guarantee:** It may fly toward terrain that looks near the goal but is hazardous (e.g., over a collapsed bridge or a storm cell), since risk/cost is not part of its decision rule.
- **Sensitivity to partial information:** If a path that looked promising becomes blocked, GBFS has no cost bookkeeping to fall back on efficiently; it can thrash between waypoints or get stuck in a greedy dead end.
- **Incompleteness on some graphs:** In graphs with cycles or repeated states it can loop unless explicit repeated-state checking is added.

ii. Behaviour of A* Search

A* evaluates nodes using f(n) = g(n) + h(n), where g(n) is the real accumulated cost (distance, weather penalty, terrain difficulty already incurred) and h(n) is the estimated remaining cost (straight-line distance to the goal). This lets the drone balance "how far have I already paid to get here" against "how much further do I estimate I must travel."

- **Cost-awareness:** A* will avoid a route that looks close in a straight line but has high real terrain/weather cost, and will prefer a slightly longer but cheaper and safer path.
- **Optimality guarantee:** If the heuristic is admissible (never overestimates remaining cost, which straight-line distance naturally satisfies when it cannot exceed true travel cost) and consistent, A* is guaranteed to return the least-cost route once one is found.
- **Adaptability to real-time updates:** When a path is reported blocked, A* can simply mark the affected edge cost as infinite/very high and re-run (or incrementally update) the search; because it tracks true cost, the re-planned route remains cost-optimal given current knowledge.
- **Trade-off:** A* explores more nodes than GBFS because it does not chase the heuristic alone, so it is somewhat slower and more memory-intensive — a real concern if the drone's onboard compute is limited and decisions must be made in milliseconds.

iii. Impact of Heuristic Accuracy

Heuristic Quality	Effect on GBFS	Effect on A*
Highly accurate (close to true cost)	Performs almost optimally and very fast, since the greedy choice usually is the right choice	Performs close to optimal with very few extra node expansions — heuristic effectively guides search directly to goal
Weak / misleading (e.g., ignores flood zones)	Can be badly misled, producing long detours or unsafe choices, with no cost mechanism to correct it	Still guaranteed optimal because g(n) compensates, but efficiency drops — more nodes must be expanded to verify true costs

Heuristic Quality	Effect on GBFS	Effect on A*
Overestimating (inadmissible)	No optimality guarantee is lost because GBFS never had one, but paths become even worse	Loses optimality guarantee — may return a route that is not the cheapest

In short: heuristic accuracy is the single most important lever for GBFS's usefulness (it lives or dies by h(n)), whereas for A* it mainly affects efficiency, not correctness — as long as admissibility is preserved.

iv. Effect of Dynamic Changes (Blocked Paths)

- **Search-tree invalidation:** A previously explored path can become invalid mid-flight, so both algorithms must support re-planning rather than a one-shot plan.
- **Re-planning cost:** GBFS re-plans cheaply but may repeatedly walk into newly-blocked routes because it has no memory of accumulated cost to prefer historically reliable paths. A* can reuse the g(n) values of unaffected nodes and typically needs to re-expand only the region downstream of the block (especially with incremental variants such as D* or Lifelong Planning A*), making it more robust to frequent map changes.
- **Oscillation risk:** Under rapidly changing blockages, greedy heuristic-only search is more prone to oscillating between waypoints, whereas A*'s cost accounting dampens this because it keeps comparing total path cost, not just proximity.
- **Necessity of an online/replanning agent:** Because information arrives only partially and changes over time, neither algorithm can be run once offline — both must be embedded in an online loop that senses, updates the cost map, and re-searches.

v. Recommendation and Justification

Recommended algorithm: A* (with a fast, admissible heuristic and incremental re-planning). Under strict real-time constraints, an implementer would typically use a weighted or bounded-suboptimal variant of A* (e.g., Weighted A* or Anytime Repairing A*) to trade a controlled, bounded amount of optimality for speed.

- **Real-time constraints:** A weighted A* can be tuned to return a “good enough, fast enough” path, unlike GBFS which is fast but unpredictable in quality.

- **Optimality/quality of delivery:** A* guarantees the least-cost (least time/least risk-weighted) path when the heuristic is admissible, which matters when "cost" encodes danger from weather/terrain, not just distance.
- **Safety:** Because A* factors in real terrain and weather cost rather than pure distance, it is far less likely to route the drone over hazardous zones purely because they look close on a straight line.

GBFS remains attractive only as a fallback when compute or time is so constrained that even A* is too slow, since it is faster but sacrifices both optimality and safety guarantees.

Scenario 2 — Smart-City Traffic Signal Optimization

Problem Formulation

This is naturally formulated as a continuous/discrete local-search (optimization) problem rather than a path-finding problem, because there is no single "goal state" to reach — instead there is a configuration space to search for the best configuration.

- **State:** A vector of signal-timing parameters (green/red/amber durations, phase offsets, cycle length) for every intersection in the city.
- **Search space:** All feasible combinations of timing parameters across hundreds of intersections — astronomically large and continuous/near-continuous.
- **Objective function:** A weighted combination to minimize total waiting time, fuel consumption, and congestion (e.g., $f = w1 \cdot \text{waitTime} + w2 \cdot \text{fuel} + w3 \cdot \text{congestionIndex}$).
- **Constraints:** Minimum/maximum phase durations, pedestrian crossing time, coordination between adjacent intersections (green waves), safety clearance intervals.

Why traditional (uninformed/informed graph) search is inefficient: classic tree/graph search algorithms (BFS, DFS, A*) are designed to find a path to a defined goal state in a state-transition graph. Here there is no discrete "goal" — we want the best point in a huge, dynamic, multi-dimensional continuous space, and the environment changes every few seconds as traffic flows. Enumerating or searching this space with path-search algorithms would be computationally intractable (curse of dimensionality) and cannot adapt continuously to changing traffic — this calls for local/metaheuristic search instead.

i. Hill Climbing in this Scenario

Hill Climbing would start from a current signal-timing configuration and iteratively make small adjustments (e.g., add two seconds of green time to one approach), keeping the change only if it improves the objective (less waiting time/congestion), and repeating until no neighbouring change improves the score.

Limitations

- **Greedy and myopic:** It only accepts improving moves, so it cannot see beyond the immediate neighbourhood of the current timing plan.
- **Gets trapped easily:** It has no mechanism to escape a locally good but globally poor configuration.
- **No exploration of far-apart configurations:** Coordinated city-wide patterns (e.g., synchronized green waves along an arterial road) may require large, coordinated changes across many intersections simultaneously — a small step at a time may never discover this.
- **Sensitive to a dynamically changing landscape:** Because traffic patterns shift continuously, the "hill" itself is moving, so a configuration that was a local optimum an hour ago may no longer be good, and Hill Climbing has no built-in adaptivity beyond re-starting.

ii. Local Maxima, Plateaus, and Ridges — Real-World Interpretation

Issue	Search Meaning	Real-World Traffic Interpretation
Local maxima	A configuration where every small change makes things worse, but it is not the best possible configuration	One intersection's timings look "optimal" in isolation, but a globally better city-wide flow (e.g., a synchronized green corridor) is never reached because it requires changing many intersections together
Plateaus	A flat region where neighbouring configurations have equal objective value	Small timing tweaks produce no measurable change in waiting time (e.g., traffic is light at that hour), so the algorithm cannot tell which

Issue	Search Meaning	Real-World Traffic Interpretation
		direction to move and may wander or stop prematurely
Ridges	A narrow path of improving states that is hard to follow using simple single-variable steps	Improvement requires simultaneously adjusting the timings of several connected intersections along a corridor in a coordinated way; single-intersection tweaks fall off the ridge and appear to worsen the objective

iii. Simulated Annealing (SA) and Genetic Algorithms (GA) as Solutions

Simulated Annealing

- **Occasionally accepts worse moves:** with a probability that decreases over time (as the "temperature" cools), allowing the search to escape local maxima and cross plateaus that pure Hill Climbing cannot.
- **Naturally suited to a changing environment:** the temperature schedule can be reset or kept "warm" to allow continual adaptation as traffic patterns shift throughout the day.

Genetic Algorithms

- **Population-based:** maintains many candidate city-wide timing plans simultaneously, so it explores diverse regions of the search space in parallel rather than following a single trajectory.
- **Crossover:** combines good sub-patterns from two parent solutions (e.g., one plan's efficient corridor timing with another plan's efficient intersection timing), which is a natural way to discover coordinated, ridge-like improvements.
- **Mutation:** randomly perturbs some timings, providing the same escape-from-local-optima benefit as SA's random moves, while preserving population diversity.
- **Parallelism and scalability:** because many candidate solutions are evaluated together, GAs scale naturally to distributed/parallel computation across many intersections.

iv. Recommendation and Justification

Recommended approach: A hybrid — Genetic Algorithm (or other population-based metaheuristic) for periodic, global re-optimization of city-wide timing plans, combined with Simulated Annealing or local Hill-Climbing-with-restarts for fast, localized real-time adjustments between the periodic re-optimizations.

- **Scalability:** GA's population-based, parallelizable nature suits the huge combinatorial search space of hundreds of intersections far better than a single-point method.
- **Adaptability:** SA's temperature-based acceptance criterion allows fast, incremental adaptation to short-term traffic fluctuations without needing to re-run a full generational GA cycle.
- **Practical deployment:** many real smart-city systems use exactly this two-tier design — a slower global optimizer that periodically updates "base" timing plans and a fast local adaptive controller that fine-tunes them in real time (often reinforcement-learning based in modern systems, which is a related but distinct approach).

Scenario 3 — Autonomous Mars Rover Exploration

i. Why an Online Search Agent (Not Offline Search)

Offline search assumes the agent knows the complete state space, action costs, and outcomes in advance, and computes an entire solution before acting. The rover, however, is exploring unknown Martian terrain: it does not have a complete map, cannot predict every hazard in advance, and can only sense its immediate surroundings. An online search agent interleaves computation and action — it takes an action, observes the result, updates its knowledge, and decides the next action — which is essential when the environment must be discovered through acting rather than known in advance.

- **Unknown environment:** the map is not available beforehand, so no complete offline plan can be computed.
- **Irreversible/costly actions:** some rover moves cannot be undone (e.g., driving into soft sand), so the agent must reason carefully about each real action, not just simulate it.
- **Communication delay:** with Earth minutes away by radio, the rover cannot wait for a fully centrally-computed plan and must make local, immediate decisions.

ii. Challenges of Partial Observability and Dynamic Environments

- **Partial observability:** limited sensor range/resolution means hazards outside sensor view (craters, buried rocks, loose regolith) are unknown until the rover gets close, so decisions are made under uncertainty about what lies ahead.
- **Risk of irreversible mistakes:** an incorrect move based on incomplete information (e.g., getting stuck, as happened historically with real rovers) may be permanent given no possibility of rescue.
- **Dynamic changes:** dust storms, shifting sand, and lighting changes (affecting camera-based sensing) mean the terrain model itself can change over time, so previously mapped areas cannot always be trusted.
- **Exploration vs. exploitation trade-off:** the rover must balance moving toward scientifically interesting sample sites (exploitation of known goals) against cautious exploration needed to build a safe understanding of the terrain.

iii. Building and Updating the Internal Model

The rover maintains an internal map (often an occupancy grid or graph of visited/known cells) that starts nearly empty. As it moves and senses (via cameras, LIDAR, or stereo vision), it adds newly observed terrain features — hazards, safe zones, slopes — to this map. Techniques used in practice include incremental/incremental-replanning search algorithms (such as LRTA* — Learning Real-Time A* — which updates and remembers the heuristic estimates of visited states so that they improve over repeated encounters), and SLAM-like techniques for building a consistent map from noisy sensor data. Each time a planned move reveals new information (e.g., a hidden ditch), the rover updates its cost/heuristic estimates for that region and re-plans its next steps rather than blindly following the original plan.

iv. Online Search vs. Uninformed and Informed Offline Search

Aspect	Uninformed Search (e.g., BFS/DFS)	Informed Search (e.g., A*)	Online Search (e.g., LRTA*)
Map knowledge required	Full map assumed known in advance	Full map + heuristic known in advance	No map assumed; built incrementally through actual movement

Aspect	Uninformed Search (e.g., BFS/DFS)	Informed Search (e.g., A*)	Online Search (e.g., LRTA*)
When planning happens	Entirely before acting	Entirely before acting	Interleaved with acting — plan, move, sense, replan
Adaptability to new hazards	None — plan is fixed	None — plan is fixed unless re-run from scratch	High — updates its knowledge and re-plans locally after every observation
Risk under partial observability	High — plan may be based on wrong assumptions	Lower than uninformed if heuristic is good, but still brittle to unknown obstacles	Lowest — decisions are always grounded in the latest sensed information

In short, uninformed and informed offline searches are appropriate when the map is fully known beforehand (e.g., planning a route on pre-surveyed terrain), but they cannot be applied directly when the map itself is being discovered — that is exactly the situation online search agents are designed for.

v. Justification of the Most Suitable Approach

Recommended approach: An online, informed search agent (e.g., LRTA* or a similar online A*-style algorithm) augmented with conservative, safety-first heuristics.

- **Uncertainty:** only an online approach can incorporate information as it is actually sensed, rather than assuming a static, fully known world.
- **Adaptability:** learning-augmented online search (LRTA*) improves its heuristic estimates the more the rover explores, so repeated traversal of a region becomes progressively more efficient — valuable for a mission lasting month or years.
- **Safety:** combining online search with a heuristic that penalizes uncertain/unseen terrain heavily (favoring cautious, incremental moves into sensed-safe areas) minimizes the risk of irreversible damage, which matters far more on Mars than raw optimality of the path.

Scenario 4 — University Exam Timetabling as a CSP

Problem Formulation

i. Variables, Domains, and Constraints

- **Variables:** one variable per exam to be scheduled, e.g., $Exam_1$, $Exam_2$, … $Exam_n$ (one variable per course/subject requiring an exam).
- **Domains:** for each exam variable, the domain is the set of possible (time-slot, hall) pairs it could be assigned to — e.g., {Mon-9am–Hall A, Mon-9am–Hall B, Mon-2pm–Hall A, …}. Domains may be restricted in advance by hall capacity or subject-specific scheduling windows.

Constraints (grouped by type):

- **Unary constraints:** an exam requiring special accommodation must be assigned only to a slot/hall that has the needed accessibility features or extra time allowance.
- **Binary constraints:** two exams taken by the same student cannot share the same time slot (no student overlap); a prerequisite subject's exam must occur before its dependent subject's exam (precedence constraint).
- **Higher-order/global constraints:** all exams assigned to the same hall in the same slot must not exceed hall capacity; a faculty member invigilating multiple exams cannot be double-booked (all-different style constraint over faculty assignments).

ii. Backtracking Search in this Scenario

Backtracking search assigns exams to time-slot/hall values one at a time (following, e.g., the order courses appear, or a smarter heuristic order), checking after each assignment whether existing constraints (no student overlap, hall capacity, precedence, faculty availability) are still satisfied. If a violation is detected, the algorithm backtracks — undoes the most recent assignment(s) — and tries the next value in the domain. This continues, depth-first, until either a complete consistent timetable is found or all possibilities are exhausted.

- **Variable ordering heuristics** (such as Minimum Remaining Values — schedule the most constrained exam, e.g., ones shared by many overlapping students, first) make backtracking far more effective in practice.
- **Value ordering heuristics** (such as Least Constraining Value) help pick a time-slot/hall assignment that leaves the most options open for the remaining unscheduled exams.

iii. Constraint Propagation (e.g., Forward Checking)

Forward checking looks ahead after each assignment: whenever an exam is assigned a time-slot/hall, it immediately removes now-inconsistent values from the domains of all as-yet-unassigned exams that share a constraint with it (e.g., removing that same slot from the domains of every other exam taken by any student enrolled in the just-scheduled exam).

- **Efficiency gain:** this prunes clearly doomed branches of the search tree early, long before a naive backtracking search would discover the conflict by brute trial and error, dramatically reducing wasted search effort as the number of courses/students grows.
- **Early failure detection:** if forward checking empties a variable's domain completely, the algorithm can backtrack immediately rather than continuing to build an assignment that is already guaranteed to fail.
- **Stronger propagation (e.g., arc consistency / AC-3)** can be layered on top of forward checking for even earlier pruning at the cost of extra computation per step — a worthwhile trade-off as the problem scales to hundreds of courses.

iv. Real-World Challenges and Best Strategy for Scalability

- **Combinatorial explosion:** the number of exam–slot–hall combinations grows extremely fast with more courses, students, and halls, making pure backtracking too slow at university scale without strong heuristics/propagation.
- **Soft constraints and preferences:** real timetabling also has soft constraints (student preference for gaps between exams, faculty preference for particular days) that a pure CSP satisfiability formulation does not naturally optimize — this often calls for a constraint optimization problem (COP) formulation with weighted soft constraints, or a local-search repair phase after an initial feasible CSP solution.
- **Dynamic changes:** late course additions, room unavailability, or accommodation requests may require re-solving parts of the timetable without disturbing everything already fixed (local repair rather than full re-solve).

Recommended strategy: Backtracking search enhanced with strong variable/value ordering heuristics (MRV, degree heuristic, least-constraining-value) plus constraint propagation (forward checking, or arc consistency for tighter pruning), and — for very large universities — a hybrid that uses this CSP core to find an initial feasible timetable and then a local-search/metaheuristic repair (e.g., simulated annealing

over the soft-constraint cost) to improve quality without re-solving from scratch. This combination scales far better than plain backtracking alone while still guaranteeing hard-constraint satisfaction.

Scenario 5 — Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. How Minimax Models the Problem

Minimax models the game as a two-player, zero-sum adversarial search over a game tree, where nodes alternate between the AI's (MAX) turn and the human opponent's (MIN) turn. The AI assumes the opponent plays optimally against it: at MAX nodes it picks the move that maximizes the evaluated outcome, and at MIN nodes it assumes the opponent picks the move that minimizes that outcome (i.e., worst-case for the AI). The algorithm recursively computes these values from the leaves (or a cutoff depth) back up to the root, and the AI selects the move at the root leading to the best minimax value — effectively planning for the worst realistic response from the opponent.

ii. Computational Challenges of Large Game Trees

- **Exponential growth:** the number of nodes grows as b^k, where b is the branching factor (number of legal moves) and k is the search depth — in a real-time strategy game with many units and actions, b can be very large, making full-depth minimax infeasible.
- **Strict time limits:** real-time play means the AI cannot search arbitrarily deep; it must return a decision within a fixed time budget, forcing a trade-off between search depth/breadth and response time.
- **Memory:** storing large trees (or transposition tables of visited states) can strain memory, especially with many possible unit/action combinations per turn.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-beta pruning keeps two bounds while traversing the tree: alpha (the best value MAX can guarantee so far) and beta (the best value MIN can guarantee so far). During the search, if at any MIN node the current value falls at or below alpha, or at any MAX node the current value rises at or above beta, the remaining branches at that node can be skipped entirely — because a rational opponent would never let play reach that branch (it is provably worse for them than an option already found elsewhere).

- **Why it works:** the pruned branches cannot change the final minimax value at the root, because they are guaranteed to be inferior to a move already discovered — so skipping them loses no decision quality while saving computation.
- **Best case:** with an optimal (or near-optimal) move ordering — e.g., trying the most promising moves first — alpha-beta effectively reduces the branching factor from b to roughly $\sqrt{b}$, allowing search to reach nearly double the depth in the same time budget.
- **Practical impact:** this lets the real-time strategy AI look further ahead within the same strict time limit, directly improving decision quality without changing the guarantee that the chosen move is minimax-optimal (relative to the search depth reached).

iv. Designing an Evaluation Function

Because the full game tree cannot be searched to terminal states within real-time limits, search is cut off at a bounded depth and a heuristic evaluation function estimates how good a non-terminal state is for the AI. A reasonable evaluation function would combine several weighted factors:

- **Material/resource balance:** difference in unit count, unit strength, and resources (gold/ore) between the AI and the opponent — a direct measure of army and economic strength.
- **Positional factors:** map control, terrain advantage (high ground, chokepoints), and proximity of forces to strategic objectives (bases, resource nodes).
- **Tempo/mobility:** number of available actions/options, unit mobility, and production capacity, reflecting future flexibility rather than just the current snapshot.
- **Threat and safety:** vulnerability of the AI's own base/units to imminent attack, weighted heavily since losses can be difficult to recover from.
- **Justification of weighting:** these factors are combined as a weighted sum (or a small learned model) so the function can be tuned/trained — factors with irreversible consequences (like losing a base) are typically weighted more heavily than easily-recoverable factors (like a temporary resource deficit), reflecting real strategic priorities.

v. Strategy for Balancing Optimality and Real-Time Performance

- **Iterative deepening with a time cutoff:** run alpha-beta at increasing depths and return the best move found so far when the time budget expires, guaranteeing a legal, reasonable move is always available even if deep search is cut short.

- **Good move ordering:** order move exploration using heuristics (e.g., previously best moves, killer-move heuristics) to maximize the pruning benefit of alpha-beta, effectively buying extra depth for the same time cost.
- **Quiescence search / selective deepening:** extend search slightly in "volatile" positions (e.g., right before or after a battle) where the evaluation function is likely to be misleading at a shallow cutoff, while keeping quiet positions shallow to save time.
- **Transposition tables and caching:** cache evaluations of previously seen states to avoid re-computing them if the same state is reached via a different move order.

Overall, the recommended strategy is time-bounded, iteratively-deepened alpha-beta search with a well-tuned, multi-factor evaluation function and strong move ordering — this gives the AI the best achievable decision quality (as close to true minimax-optimal as the time budget allows) while strictly respecting the real-time constraints of competitive play.

Summary Table — Algorithm Choice by Scenario

Scenario	Core Technique(s)	Key Reason for Choice
1. Drone delivery	A* (weighted/anytime variant) with incremental re-planning	Balances real path cost and heuristic; guarantees quality routes under safety and time pressure
2. Traffic signal optimization	Genetic Algorithm (global) + Simulated Annealing (local)	Population-based global search scales to huge space; SA adapts quickly to short-term changes
3. Mars rover exploration	Online informed search (e.g., LRTA*)	Builds knowledge as it senses; only viable approach when the map is unknown and partially observable
4. Exam timetabling	Backtracking + Forward Checking (with MRV/LCV heuristics)	Guarantees hard-constraint satisfaction while pruning

Scenario	Core Technique(s)	Key Reason for Choice
		the search space efficiently as scale grows
5. Game AI	Minimax + Alpha-Beta Pruning + Iterative Deepening	Provides near-optimal adversarial decisions within strict real-time limits